An Equivalent Axiomatization of ZF via a Single *Closure* Schema

Mathematics and a Machine-Verified Rocq/Coq Proof

Vladimir Reshetnikov

Formalization carried out with Claude Code (Rocq 9.0.1)

2026-06-30, revised 2026-07-01

Abstract

We study an alternative axiomatization T of Zermelo–Fraenkel set theory obtained by deleting the four "generative" axioms — Pairing, Union, Infinity, and Replacement — and adding in their place a single new schema, *Closure*: the transitive closure of any set under any *set-like* definable relation is a set. Keeping Extensionality, Regularity, Separation, and Powerset, we show that T is *exactly* ZF. (Choice plays no part in the trade and is omitted from both theories; adding it back, $\mathsf{T} \cup \{\mathsf{C}\}$ and ZFC coincide.) The forward half (the four deleted axioms are theorems of T) is the interesting direction; it rests on one small fact — every set has a host, i.e. is a member of some set — which turns Closure into a fully general principle of collection. The reverse half (every ZF model satisfies Closure) is the textbook transitive-closure recursion. We then describe in tutorial detail a complete machine-checked formalization in the Rocq/Coq proof assistant, organized as a small reusable library: a shallow embedding proving the semantic equivalence in both directions with a "free dependency audit" that certifies *which* axioms each derivation actually uses; a deep embedding of first-order logic that re-proves the forward trade from genuinely syntactic schemas; a sound natural-deduction calculus with the cross-theory corollary $\mathsf{ZF} \vdash \varphi \Rightarrow \mathsf{T} \models \varphi$; a from-scratch proof of Gödel completeness, compactness for sentence theories, and the forward *syntactic* direction $\mathsf{ZF} \vdash \varphi \Rightarrow \mathsf{T} \vdash \varphi$; and finally the *deep reverse*: the finite recursion theorem rendered with genuine first-order formulas and carried out inside an arbitrary first-order ZF-model, which yields the converse $\mathsf{T} \vdash \varphi \Rightarrow \mathsf{ZF} \vdash \varphi$ and hence the full machine-checked deductive equivalence $\mathsf{T} \vdash \varphi \Leftrightarrow \mathsf{ZF} \vdash \varphi$. The whole development contains no admitted goals and rests only on the standard axioms of classical mathematics.

Contents

1 Introduction

The usual axiomatization of Zermelo–Fraenkel set theory carries several *generative* axioms whose common job is to assert that certain sets exist: **Pairing** ($\{a, b\}$ exists), **Union** ($\bigcup s$ exists), **Infinity** (an inductive set exists), and **Replacement** (the image of a set under a definable function-class is a

set). Each of these is, in isolation, a recipe for collecting a previously described family of sets into one object.

This article studies what happens when we replace all four with a *single* collection principle. Keep the "structural" axioms — Extensionality, Regularity, Separation, and Powerset — and add the following schema.

> **Closure.** For every *set-like* definable binary relation $\prec$ and every set s, there is a set w that contains s and is closed under taking $\prec$-predecessors. (Such a w is a superset of the transitive closure of s under $\prec$.)

Here "set-like" is the natural smallness condition that makes the principle consistent: each node's class of immediate $\prec$-predecessors must already be bounded by some set. Without it, "$z \prec x$" could be "$z = z$", and the closure of $\{\emptyset\}$ would be the universe.

Call the resulting theory T. The thesis of this article — proved on paper and then in full inside a proof assistant — is:

> *T and ZF have exactly the same theorems.*

A word on Choice. We work throughout in ZF rather than ZFC. The Axiom of Choice plays no part in the trade between the four generative axioms and the Closure schema — neither direction uses it — so we omit it from both theories and state everything for ZF and T. Adding it back is harmless and symmetric: since T and ZF prove the same theorems, $\mathsf{T} \cup \{\mathsf{C}\}$ and ZFC also coincide (Corollary 5.2). By the Axiom of Choice we mean any of its standard equivalents, for instance *"the Cartesian product of any collection of nonempty sets is nonempty."* One caveat, returned to in §6 and §11: the *proof assistant* still uses a metatheoretic description operator (Hilbert's ε) to name objects asserted to exist. That is choice in the ambient logic we reason *in*, not the set-theoretic axiom we are setting aside; the two are independent, and we flag the metatheoretic use wherever it occurs.

What makes this more than a bookkeeping exercise is the *forward* direction: deriving Pairing, Union, Infinity, and Replacement from the lone Closure schema. We will see that all four are instances of *one* idea, and that the engine which makes them go is not Closure by itself but Closure standing on a single, almost-trivial principle of **hosting** — that every set is a member of some set:

$$\forall a \, \exists y \; (a \in y).$$

(Powerset supplies it, since $a \subseteq a$ gives $a \in \mathcal{P}(a)$, but hosting is far weaker — see §3.3.) Hosting is what lets us certify that the relations we care about are set-like, and hence what upgrades Closure from a statement about transitive closures into a fully general principle of collection.

What this article does. There are two interleaved goals. The first is mathematical: state the two theories precisely, prove the equivalence, exhibit the four derivations as one schema-instance family, and isolate what genuinely powers the trade — the *hosting* principle, a trivial shadow of Powerset — while showing that Powerset in full, Regularity, and Choice are all more than the trade needs (Regularity and Choice being used in neither direction). The second is to explain, in tutorial style, the *machine-verified* proof we built in the Rocq/Coq proof assistant. That formalization grew well past the original equivalence: it now includes a deep embedding of first-order logic, a sound and *complete* proof calculus (Gödel completeness proved from scratch), a compactness theorem for sentence theories, the forward syntactic statement $\mathsf{ZF} \vdash \varphi \Rightarrow \mathsf{T} \vdash \varphi$, and — via the recursion theorem built as genuine first-order formulas inside an arbitrary ZF-model — the converse, closing

the machine-checked equivalence $\mathsf{T} \vdash \varphi \Leftrightarrow \mathsf{ZF} \vdash \varphi$. We will walk through each layer, quote the actual definitions and theorem statements, and explain the design decisions and how the one genuine obstacle — the deep reverse — was eventually overcome.

Roadmap. Sections 2–5 are the mathematics: the two theories, the forward direction (with the "how much Powerset?" analysis), the reverse direction, and the equivalence theorem. Sections 6–9 are the formalization: the shallow embedding and the dependency audit (§6), the deep embedding closing the first-order gap (§7), the proof calculus and soundness (§8), and completeness, compactness, and the forward syntactic direction (§9). Section 10 is the deep reverse: the recursion theorem as a first-order formula, the converse syntactic direction, and the full equivalence. Section 11 states precisely what is proven. Section 12 reflects on both punchlines.

2 The two theories

We work in the first-order language with one binary relation symbol $\in$ and equality. All schemas allow parameters (free variables other than the displayed ones). Throughout, $\subseteq$ abbreviates the definable inclusion $a \subseteq b \coloneqq \forall x\,(x \in a \to x \in b)$.

2.1 The common base

Both theories contain:

Extensionality
$\forall a\,\forall b\,\big(\forall x\,(x \in a \leftrightarrow x \in b) \to a = b\big)$.

Separation (schema)
for each formula $\varphi(x, \bar{p})$, $\forall \bar{p}\,\forall a\,\exists s\,\forall x\,\big(x \in s \leftrightarrow (x \in a \wedge \varphi(x, \bar{p}))\big)$.

Powerset
$\forall a\,\exists p\,\forall x\,(x \in p \leftrightarrow x \subseteq a)$.

Regularity
every nonempty set has an $\in$-minimal element.

(We work in ZF, so Choice is not part of the common base; see the note in §1 and Corollary 5.2 for the ZFC statement.)

2.2 The theory ZF

ZF adds the four generative axioms:

Pairing
$\forall a\,\forall b\,\exists p\,\forall x\,(x \in p \leftrightarrow (x = a \vee x = b))$.

Union
$\forall s\,\exists u\,\forall x\,\big(x \in u \leftrightarrow \exists v\,(x \in v \wedge v \in s)\big)$.

Infinity
there is a set containing $\emptyset$ and closed under $x \mapsto x \cup \{x\}$.

Replacement (schema)
for each functional formula, the image of a set is a set.

2.3 The theory T

T drops all four and adds the single **Closure** schema. Two definitions make it precise. A binary relation $\prec$ is *set-like* when

$$\forall x\, \exists y\, \forall z\, (z \prec x \to z \in y), \tag{1}$$

i.e. the class of immediate $\prec$-predecessors of each node is bounded by a set. A set w is *closed under $\prec$-predecessors above s* when

$$s \subseteq w \ \wedge\ \forall u\, \forall v\, (u \prec v \wedge v \in w \to u \in w). \tag{2}$$

The Closure schema asserts one such w for every set-like definable $\prec$ and every s:

$$\underbrace{\big(\forall x\, \exists y\, \forall z\, (z \prec x \to z \in y)\big)}_{\prec \text{ set-like}} \implies \forall s\, \exists w\, \underbrace{\big(s \subseteq w \ \wedge\ \forall u\, \forall v\, (u \prec v \wedge v \in w \to u \in w)\big)}_{w \text{ contains the transitive closure of } s}. \tag{3}$$

There is one instance of (3) per definable binary relation $\prec$, parameters allowed.

Remark 2.1 (Why w is a *superset*, not the exact transitive closure)**.** We deliberately ask only for a set w *containing* the transitive closure, not for the transitive closure itself. This is harmless: once w exists, Separation carves out the exact transitive closure as a subset of w. Asking only for a superset keeps each Closure instance about an existential "there is a bounding set," which is what makes the set-like hypothesis exactly the right amount of input.

Remark 2.2 (The trade is purely "generative")**.** Regularity is shared identically by both theories and (as the audit in §6 certifies) is used in *neither* direction of the trade. The equivalence therefore reduces to the purely "generative" content: over the common base $\{\text{Ext}, \text{Sep}, \text{Pow}\}$ (with Regularity along for the ride), the four axioms $\{\text{Pairing}, \text{Union}, \text{Infinity}, \text{Replacement}\}$ are interderivable with the single schema Closure.

3 The forward direction: T $\vdash$ ZF

This is the interesting half. We assume only $\{\text{Extensionality}, \text{Separation}, \text{Powerset}, \text{Closure}\}$ and derive Pairing, Union, Replacement, and Infinity. (Regularity is not needed here; it is a genuine passenger, as the dependency audit in §6 will certify.)

3.1 The linchpin: every set has a host

The engine of the forward direction is one almost-trivial fact: every set is a *member* of some set. Call a set y a *host* of a when $a \in y$. As we will see, the trade needs nothing about a beyond its *having* a host.

Powerset is one ready source of hosts:

Lemma 3.1 (self-membership in the powerset)**.** *For every set a, $a \in \mathcal{P}(a)$.*

Proof. $a \subseteq a$ holds trivially, and by Powerset $x \in \mathcal{P}(a) \leftrightarrow x \subseteq a$; take $x = a$. □

It is worth seeing how little of Powerset this uses. Consider any class relation $\prec$ that is *singleton-valued* above each node — i.e. for each x there is at most one z with $z \prec x$, given by some class function $z = G(x)$. Is such a $\prec$ set-like? The set-like condition (1) asks for a set y bounding $\{\, z : z \prec x \,\} = \{G(x)\}$ — that is, a *host of* $G(x)$. Lemma 3.1 supplies one ($\mathcal{P}(G(x))$), but any host would do; the powerset is never opened up. More generally, whenever the predecessor-class above each node already sits inside some set, that set is the bound.

This is the conceptual heart of the whole equivalence: ***hosting — "every set is a member of some set" — turns "has a definable predecessor" into "is set-like."*** *Closure then collects the predecessors. The four generative axioms are four ways of choosing the relation $\prec$ and the seed s. We will see in §3.3 that hosting is* all *that is used, so full Powerset is more than the trade needs.*

3.2 Four derivations, one schema-instance family

Axiom	Relation $\prec$ (read $z \prec x$)	Seed s
Union	$z \in x$ (membership)	the given set s
Replacement	$x \in a \wedge z = F(x)$ (function graph)	the domain a
Pairing	$(x{=}\emptyset \wedge z{=}a) \vee (x{=}\{\emptyset\} \wedge z{=}b)$	the 2-node seed $\{\emptyset, \{\emptyset\}\}$
Infinity	$z = x \cup \{x\}$ (successor)	$\{\emptyset\}$

In each case the recipe is the same: (i) check the relation is set-like (using Lemma 3.1); (ii) apply Closure to get a closed superset w; (iii) use Separation to carve the exact set we wanted out of w.

Union is closure under $\in$. The relation $z \prec x \coloneqq z \in x$ is set-like because the predecessors of x are exactly the elements of x, bounded by x itself ($y = x$). Closure applied to s gives a w with $s \subseteq w$ and w closed under $\in$-predecessors. Hence every element of every member of s lies in w, and Separation extracts $\{\, x \in w : \exists v\, (x \in v \wedge v \in s)\,\} = \bigcup s$.

Replacement is closure along a function graph. For a class function F and domain a, take $z \prec x \coloneqq x \in a \wedge z = F(x)$. The predecessors of x form the singleton $\{F(x)\}$ (or $\emptyset$ if $x \notin a$), bounded by any host of $F(x)$ (Lemma 3.1 gives one, $\mathcal{P}(F(x))$) — this is where hosting is used. Closure applied to a yields a w containing the whole image $\{F(x) : x \in a\}$, and Separation carves out exactly that image.

Pairing is the subtle one, and it is precisely where the host-providing role becomes visible as a *design constraint*. We want $\{a, b\}$ for arbitrary a, b. The naive attempt — one seed node whose predecessors are $\{a, b\}$ — is self-defeating: to certify that relation set-like we would need a set bounding $\{a, b\}$, which is Pairing itself. The fix is to *split a and b across two different seed nodes.* Use the two-node seed $\{\emptyset, \{\emptyset\}\}$ (a fixed two-element set, built without Pairing) and the relation

$$z \prec x \;\coloneqq\; (x = \emptyset \wedge z = a) \;\vee\; (x = \{\emptyset\} \wedge z = b).$$

Now each node has a *singleton* predecessor-class — $\{a\}$ above $\emptyset$, $\{b\}$ above $\{\emptyset\}$ — so hosting makes it set-like (any hosts of a and b work — e.g. $\mathcal{P}(a)$, $\mathcal{P}(b)$), with no circular appeal to Pairing. Closure of the seed then drags both a and b into one set w, and Separation gives $\{x \in w : x = a \vee x = b\} = \{a, b\}$.

Infinity is closure of $\{\emptyset\}$ under the successor relation $z \prec x \coloneqq z = x \cup \{x\}$. The successor is unique, so the predecessor-class is again a singleton, hostable (any host of the successor works); the relation is set-like. Closure applied to $\{\emptyset\}$ gives a set w containing $\emptyset$ and closed under successors — an inductive set, which is exactly what Infinity asserts. (Once Pairing and Union are available, the successor $x \cup \{x\}$ is itself a set, so the relation is genuinely definable.)

3.3 How much Powerset? Only hosting

The four derivations use Powerset in a single, uniform role: as a source of hosts in Lemma 3.1, the fact that makes singleton-valued (and more generally suitably bounded) relations set-like. Union is the lone exception — its relation $z \in x$ is bounded by x directly, with no host needed — which is why the audit in §6 shows Union depending on Separation and Closure *only*.

This answers a natural question: *do we need the full Powerset axiom?* No. The trade never opens a powerset up; it uses only the consequence

$$\text{(H)} \qquad \forall a\, \exists y\, (a \in y) \qquad (\text{“every set is a member of some set”}),$$

the *hosting* principle. Powerset yields (H) via $a \in \mathcal{P}(a)$, but (H) is far weaker — it holds, for instance, in $H(\aleph_1)$, the hereditarily countable sets, which has no $\mathcal{P}(\omega)$. The machine confirms it: with the hypothesis weakened from Powerset to (H), all four derivations still go through, and `Check` reports them depending on (H), not on Powerset (§6).

Remark 3.2 (Would *finite* Powerset suffice? No)**.** A tempting weakening is “Powerset restricted to finite sets” (under any reasonable notion of finite — Dedekind, Tarski). It is not enough. The predecessors we must host — a, b in Pairing, $F(x)$ in Replacement, $x \cup \{x\}$ in Infinity — are *arbitrary*, possibly infinite sets, and finite Powerset produces $\mathcal{P}(a)$ only for finite a, so it cannot host an infinite set; it does not yield (H). Hosting infinite sets is moreover unavoidable: deriving Pairing of two infinite sets forces, through set-likeness, a bounding set containing each of them, and the base {Extensionality, Separation, Closure} has no other way to manufacture a host (Closure cannot host a set it is not already handed, on pain of circularity). Indeed Pairing already implies (H) — $a \in \{a, b\}$ — so any sufficient fragment must prove (H), which finite Powerset does not. The right way to weaken Powerset here is therefore not “on finite sets” but “replace it by hosting.”

So, on the structural side, the picture is uniform: *none* of Powerset, Regularity, or Choice is used in full. Regularity and Choice are passengers, used in neither direction (§4); Powerset is used only through its trivial hosting shadow (H), and only forward (the reverse needs no Powerset at all). The genuine engine of the equivalence is just {Extensionality, Separation, Closure} together with hosting.

4 The reverse direction: ZF ⊢ Closure

The reverse direction is the standard transitive-closure construction; the only care needed is in collecting an ω-indexed family of sets into one object, which is exactly the job Replacement and Infinity exist to do.

Fix a set-like relation R and a set s. Define the one-step operator

$$g(t) \;=\; t \;\cup\; \{\, u : \exists v \in t,\; u\, R\, v \,\}.$$

The bracketed predecessor-set is genuinely a set. Set-likeness says that for each v there *exists* a set bounding $\{u : u\, R\, v\}$; the *Collection* schema — a theorem of ZF (provable from Replacement together with Foundation, and itself choice-free) — gathers these into a single set B holding a bound for every $v \in t$, so $\bigcup B$ bounds all predecessors at once and Separation carves out the exact predecessor-set. (Collection delivers a bounding *set*, not a chosen function, so no Choice is needed — cf. the note in §1.) So $g : V \to V$ is a definable total operation.

Now iterate: $W_0 = s$ and $W_{n+1} = g(W_n)$, indexing on the *metatheory's* natural numbers. The closure we want is $w = \bigcup_n W_n$. To produce w as a single set we must turn the meta-indexed

family $\{W_n\}$ into an *object*-indexed family. Use Infinity to obtain an inductive set, and inside it the von Neumann numerals $\underline{n}$ ($\underline{0} = \emptyset$, $\underline{n+1} = \underline{n} \cup \{\underline{n}\}$). The assignment $\underline{n} \mapsto W_n$ is a definable class function on the numerals, so Replacement makes its image $\{W_n : n\}$ a set, and Union gives $w = \bigcup_n W_n$.

For this to be well defined we need the numerals to be *distinct*, i.e. $\underline{m} = \underline{n} \Rightarrow m = n$, so that "$\underline{n} \mapsto W_n$" is unambiguous. Distinctness follows from $\underline{m} \in \underline{n}$ whenever $m < n$, together with the fact that no *numeral* is a member of itself. One might reach for Regularity here (apply it to $\{\underline{n}\}$), but that is a *convenience*, not a necessity. The finite von Neumann numerals are non-self-membered already in ZF *without* Foundation: each $\underline{n}$ is a genuine finite ordinal — a transitive set strictly well-ordered by $\in$ — and an ordinal is never a member of itself (that would violate the irreflexivity of its own $\in$-ordering), a fact established by a plain induction on n rather than by the global Foundation axiom. So Regularity is *not* needed for the reverse direction; we make this precise, and let the machine certify it, in §6. Finally, w contains $s = W_0$ and is closed under R-predecessors: if $u\ R\ v$ and $v \in w$, then $v \in W_n$ for some n, hence $u \in g(W_n) = W_{n+1} \subseteq w$.

Remark 4.1 (Regularity is a passenger, in both directions)**.** What actually powers the reverse construction is Replacement, Union, and Infinity (to collect the family) — not Powerset, and (by the Foundation-free argument above) not Regularity either. The machine audit in §6 confirms both omissions, yielding the sharper statement

$$\mathsf{ZF} - \text{Powerset} - \text{Regularity} \ \vdash\ \text{Closure}.$$

Together with the forward audit (which uses neither Regularity nor Choice), this says that **Regularity, like Choice, is a passenger of the whole trade**: the interderivability of Closure with {Pairing, Union, Infinity, Replacement} holds over the bare base {Extensionality, Separation, Powerset}, with Regularity and Choice added equally to both sides and used by neither. The one axiom that is genuinely *asymmetric* is Powerset — and even it is used forward only through its hosting shadow (§3.3) and is idle in reverse.

5 The equivalence theorem

Theorem 5.1 (Equivalence)**.** *Over the base {Extensionality, Separation, Powerset}, the schema Closure is interderivable with {Pairing, Union, Infinity, Replacement}. Consequently — adding Regularity to both sides, where it is used by neither — the theory* T *(adding Closure) and the theory* ZF *(adding Pairing, Union, Infinity, Replacement) have exactly the same theorems.*

Proof. $\mathsf{T} \vdash \mathsf{ZF}$: Pairing, Union, Replacement, and Infinity are Theorems by the four derivations of §3, none of which uses any ZF-only axiom. $\mathsf{ZF} \vdash \mathsf{T}$: Closure is a Theorem by the construction of §4. Since the two theories share their remaining axioms verbatim, they prove the same sentences. □

Corollary 5.2 (Adding Choice)**.** $\mathsf{T} \cup \{\mathsf{C}\}$ *and* ZFC *have exactly the same theorems.*

Proof. Choice is the *same* sentence C adjoined to both theories. For any φ, the deduction theorem gives $\mathsf{T} \cup \{\mathsf{C}\} \vdash \varphi$ iff $\mathsf{T} \vdash (\mathsf{C} \to \varphi)$ iff (Theorem 5.1) $\mathsf{ZF} \vdash (\mathsf{C} \to \varphi)$ iff $\mathsf{ZFC} \vdash \varphi$. The argument is insensitive to which standard form of C one adopts. □

The rest of the article is about making Theorem 5.1 — and a good deal more — checkable by machine.

6 Mechanization I: the shallow embedding and the dependency audit

The formalization is organized by topic in three repository subtrees: `Logic/FirstOrder/Coq/`, `SetTheory/ZF/Coq/`, and `SetTheory/ClosureAxiomatization/Coq/`. Together they contain seven Rocq/Coq source files organized as a small library (see the appendix for the module map). This section covers the two self-contained files that prove the semantic equivalence directly: `Forward.v` and `Reverse.v`.

6.1 Why Rocq/Coq, and what a "shallow embedding" is

We use Rocq/Coq (version 9.0.1). The decisive feature is the **Section/Hypothesis** idiom, which fits our problem like a glove. A Coq `Section` lets us fix an abstract structure and assume axioms about it as local `Hypothesis`es; every theorem proved inside is, after the section closes, automatically generalized over the structure *and over exactly the hypotheses it used*. That is precisely the shape of a metatheorem of the form "every model of these axioms satisfies that conclusion."

The structure is modelled *shallowly*: the universe of sets is an abstract Coq type `V`, membership is an abstract relation `mem : V -> V -> Prop`, and the axiom schemas are rendered with the metatheory's own predicates. For instance Separation quantifies over a Coq predicate `P : V -> Prop`:

```
Variable V : Type.
Variable mem : V -> V -> Prop.
Infix " " := mem (at level 70, no associativity).
Definition Sub (a b : V) : Prop := forall x, x  a -> x  b.

Hypothesis Extensionality :
  forall a b, (forall x, x  a <-> x  b) -> a = b.
Hypothesis Separation :
  forall (a : V) (P : V -> Prop),
    exists s, forall x, x  s <-> (x  a /\ P x).
Hypothesis Powerset :
  forall a, exists p, forall x, x  p <-> Sub x a.
```

The Closure schema becomes a quantification over a Coq relation `R : V -> V -> Prop`, with set-likeness defined exactly as in (1):

```
Definition SetLike (R : V -> V -> Prop) : Prop :=
  forall x, exists y, forall z, R z x -> z  y.

Hypothesis Closure :
  forall R : V -> V -> Prop, SetLike R ->
    forall s, exists w, Sub s w /\ (forall u v, R u v -> v  w -> u  w).
```

What a shallow embedding proves, and its faithfulness. Because the schemas range over *all* Coq predicates/relations, the artifact establishes the *second-order* statement: every structure $(V, \in)$ satisfying these axioms also satisfies the derived ones. This is the standard, accepted way to mechanize an equivalence of axiomatizations, and it is strictly stronger than the first-order schematic reading on the model side. It stays faithful to the genuine first-order claim because *every derivation*

instantiates a schema at one concrete, definable relation — the membership relation, a function graph, the two-branch pairing relation, the successor relation — exactly the instances a first-order proof would write down. (Section 7 removes even this caveat for the forward direction, by re-proving it with schemas that range over syntactic formulas.)

6.2 The linchpin and the four theorems, in Coq

The linchpin is two lines of Coq — the `host` operator (from the `Hosting` hypothesis $\forall a\, \exists y\, a \in y$, packaged by description) and its spec — exactly the pivot the informal argument advertised:

```
(* THE LINCHPIN, minimal form: every set has a host (a  host a). *)
Definition host (a : V) : V :=
  proj1_sig (constructive_indefinite_description _ (Hosting a)).
Lemma host_spec : forall a, a  host a.
Proof. intro a. exact (proj2_sig (constructive_indefinite_description _ (Hosting
    a))). Qed.
```

A side lemma `powerset_gives_hosting` records that Powerset satisfies `Hosting` ($a \in \mathcal{P}(a)$) — the only place the file touches the Powerset hypothesis at all.

The four relations of the schema-instance family are declared verbatim:

```
Definition pairRel (a b : V) : V -> V -> Prop :=
  fun z x => (x = emptyset /\ z = a) \/ (x = single_empty /\ z = b).
Definition memRel : V -> V -> Prop := fun z x => z  x.
Definition graphRel (F : V -> V) (a : V) : V -> V -> Prop :=
  fun z x => x  a /\ z = F x.
Definition succRel : V -> V -> Prop :=
  fun z x => forall t, t  z <-> (t  x \/ t = x).
```

Each of Pairing, Union, Replacement, Infinity is then proved by the three-step recipe (set-like check → Closure → Separation). Union is the shortest and shows the pattern cleanly:

```
Theorem Union :
  forall s, exists u, forall x, x  u <-> exists v, x  v /\ v  s.
Proof.
  intro s.
  assert (HSL : SetLike memRel).
  { intro x. exists x. intros z Hz. unfold memRel in Hz. exact Hz. }
  destruct (Closure memRel HSL s) as [w [Hsub Hclosed]].
  exists (sep w (fun x => exists v, x  v /\ v  s)).
  intro x. split.
  - intro H. exact (sep_elim2 _ _ _ H).
  - intro H. apply sep_intro.
    + destruct H as [v [Hxv Hvs]]. apply (Hclosed x v).
      * unfold memRel. exact Hxv.
      * apply Hsub. exact Hvs.
    + exact H.
Qed.
```

Read it as English: "`memRel` is set-like (host $= x$ itself); apply `Closure` to get a closed superset w of s; separate out the genuine union elements; membership in either direction is immediate from closure/Separation."

The Pairing proof is where the two-node seed appears and where the set-like check does real work: it splits on whether the node is $\emptyset$ or $\{\emptyset\}$ and supplies `host a` or `host b` accordingly — the Coq transcription of the "split a and b across different nodes" argument from §3. (The seed $\{\emptyset, \{\emptyset\}\}$ is itself built from `host` and a one-edge Closure, not from Powerset.)

6.3 The free dependency audit

Here is the part that turns the "only hosting is used" claim from rhetoric into a machine certificate. When a Coq `Section` closes, each theorem is generalized over precisely the hypotheses it referenced; the trailing `Check` commands print those generalized types, so we can simply *read off* which axioms each derivation used:

```
End ClosureEquivalence_Forward.
Check Pairing.
Check Union.
Check Replacement.
Check Infinity.
```

The printed signatures certify:

- **Union** depends on *Separation and Closure only* — not Hosting, not Powerset, not Extensionality, not the nonempty-domain assumption. (Its relation $z \in x$ is self-bounding.)
- **Replacement** depends on *Separation, Hosting, and Closure* — Hosting in exactly its host-providing role, and *not* Powerset (nor Extensionality, nor nonemptiness).
- **Pairing** depends on *Separation, Hosting, Closure* and a *nonempty domain* (to build $\emptyset$) — and, now that the seed is built by Closure rather than by a powerset, not even Extensionality.
- **Infinity** additionally needs *Extensionality* (for the inductive-set uniqueness step).
- **Powerset** is referenced by *none* of the four (only by the side lemma `powerset_gives_hosting`); **Regularity** by none either — both, with Choice, are genuine passengers.

The same audit on `Reverse.v` (`Check Closure_holds`) shows `Closure_holds` depending on a nonempty domain, Extensionality, Separation, Pairing, Union, Infinity, and Replacement — but **neither Powerset nor Regularity**. This is the machine confirming the sharper $\mathsf{ZF} -$ Powerset $-$ Regularity $\vdash$ Closure of Remark 4.1, and with it the asymmetry: Powerset is used forward (only through hosting, §3.3) and idle in reverse, while Regularity is idle in *both* directions — a passenger of the trade alongside Choice.

6.4 The reverse construction in Coq

`Reverse.v` transcribes §4 directly. The meta-level iteration is an ordinary Coq `Fixpoint` on `nat`:

```
Fixpoint iterate (g : V -> V) (s : V) (n : nat) : V :=
  match n with O => s | S k => g (iterate g s k) end.
```

The numerals' distinctness is established *without* Regularity: each numeral is a transitive set (`onat_trans`), hence non-self-membered (`onat_no_self`), both by induction on `nat`, from which injectivity follows:

```
Lemma onat_trans   : forall n x, x   onat n -> Sub x (onat n).   (* Foundation-free *)
Lemma onat_no_self : forall n, ~ onat n   onat n.                (* no Regularity *)
Lemma onat_inj     : forall i j, onat i = onat j -> i = j.
```

`onat_inj` is what makes the map "numeral $\mapsto$ iterate" well defined, so Replacement can collect $\{W_n\}$ into a set and Union flatten it. Because this path no longer touches Regularity, `Check Closure_holds` drops it from the dependency list (Remark 4.1). The headline theorem is then

```
Theorem Closure_holds :
  forall R : V -> V -> Prop, SetLike R ->
    forall s, exists w, Sub s w /\ (forall u v, R u v -> v   w -> u   w).
```

with $w = \bigcup_n W_n$ realized as `ounion (imageR (Ffun R HSL s) Inf)` — the object union of the Replacement-image, over the inductive set `Inf`, of the numeral-to-iterate map `Ffun`.

6.5 Where classical logic — and metatheoretic choice — enter

Both files import `ClassicalEpsilon`. It is used to *package* the existential axioms as operations — turning "$\exists p, \ldots$" (Powerset) into a function `power : V -> V` with a specification lemma, via `constructive_indefinite_description` — and, in `Reverse.v`, to extract a bounding function `boundf` from set-likeness and to index the iteration.

It is worth being precise about what kind of choice this is, since we went to some trouble in §2 to keep the *set-theoretic* Axiom of Choice out of both theories. The description operator here is choice in the *ambient logic* we reason *in* — it only gives a name to an object the theory already asserts to exist — and it is independent of the object-level Axiom of Choice, present in both directions regardless. In particular `boundf` is a *convenience*: as §4 explained, the underlying ZF mathematics needs only the choice-free Collection schema to form the predecessor-set, so no set-theoretic Choice is smuggled in through the back door. The `Print Assumptions` audit in §11 confirms that neither direction depends on an object-level Choice axiom.

7 Mechanization II: closing the first-order gap

The shallow embedding's one honest caveat is that its schemas range over arbitrary Coq predicates rather than over first-order *formulas*. The library removes that caveat by building a genuine deep embedding of the first-order language of set theory (`Fol.v` — syntax, renaming, free variables, sealing, a formula enumeration, and Tarski satisfaction over an arbitrary structure) and re-proving the trade from schemas that quantify over syntactic formulas (the `DeepForward` section of `Equivalence.v`).

7.1 Syntax, environments, and Tarski satisfaction

A formula is an element of an inductive type, with De Bruijn indices for variables (so a variable is a `nat` counting binders outward, and α-equivalence is syntactic equality):

```
Inductive form : Type :=
| fMem : nat -> nat -> form
| fEq  : nat -> nat -> form
| fBot : form
| fImp : form -> form -> form
| fAnd : form -> form -> form
```

```
| fOr  : form -> form -> form
| fAll : form -> form
| fEx  : form -> form.
```

An *environment* `e : nat -> V` assigns a domain element to each variable. `scons d e` pushes a new binding `d` at index 0 and shifts the rest — this is how a quantifier extends the environment under its binder. Tarski satisfaction is then a straightforward structural recursion:

```
Definition scons (d : V) (e : nat -> V) : nat -> V :=
  fun n => match n with O => d | S k => e k end.

Fixpoint Sat (e : nat -> V) (f : form) {struct f} : Prop :=
  match f with
  | fMem i j => (e i)  (e j)
  | fEq i j  => e i = e j
  | fBot     => False
  | fImp a b => Sat e a -> Sat e b
  | fAnd a b => Sat e a /\ Sat e b
  | fOr a b  => Sat e a \/ Sat e b
  | fAll a   => forall d, Sat (scons d e) a
  | fEx a    => exists d, Sat (scons d e) a
  end.
```

Two infrastructural lemmas carry the whole development. `Sat_ext` says satisfaction depends only on the environment's values on the variables that occur; `Sat_rename` relates satisfaction of a renamed formula to satisfaction under the renamed environment:

$$\mathrm{Sat}\ e\ (\texttt{rename}\ r\ f)\ \leftrightarrow\ \mathrm{Sat}\ (e \circ r)\ f.$$

Renaming is the deep-embedding analogue of variable substitution; because our signature is *purely relational*, there are no function symbols and hence no compound terms — a "term" is just a variable — so substitution of a variable for a variable is exactly a renaming. This single observation keeps the entire metatheory free of a general substitution operation, which is usually the most painful part of a deep embedding.

7.2 First-order Separation and Closure

Now the schemas range over formulas. A formula `phi` with free variable 0 denotes a unary predicate (its other free variables are parameters, read from the environment `e`); a formula `psi` with free variables $0, 1$ denotes a binary relation `relOf psi e`:

```
Hypothesis SeparationFO :
  forall (phi : form) (e : nat -> V) (a : V),
    exists s, forall x, x  s <-> (x  a /\ Sat (scons x e) phi).

Definition relOf (psi : form) (e : nat -> V) : V -> V -> Prop :=
  fun z x => Sat (scons z (scons x e)) psi.

Hypothesis ClosureFO :
  forall (psi : form) (e : nat -> V),
    SetLike (relOf psi e) ->
    forall s, exists w, Sub s w /\ (forall u v, relOf psi e u v -> v  w -> u  w).
```

`Equivalence.v` then re-derives Pairing, Union, first-order Replacement, and Infinity from *these* schemas. The new obligation, compared to the shallow version, is that each relation used must be exhibited as a concrete `form` and its `Sat`-meaning verified. For the simple cases the defining formula is short; for Replacement, expressing "$\exists x \in a,\ \psi(y, x)$" as a formula requires shuffling De Bruijn indices, which is exactly what `Sat_rename` is for. The upshot is a *formal* certificate — not a meta-remark — that the schema-trade uses only first-order definable instances:

```
Theorem Pairing       : forall a b, exists p, forall x, x   p <-> (x = a \/ x = b).
Theorem Union         : forall s, exists u, forall x, x   u <-> exists v, x   v /\ v
    s.
Theorem ReplacementFO : (* image of a set under a formula-defined functional relation
    *)
Theorem Infinity      : (* an inductive set *)
```

Why the reverse direction is harder to deepen. The forward trade ports to the deep setting cheaply because every relation it uses is a short first-order formula. The reverse direction does *not*: its essential use of Replacement collects the family $\{W_n\}$ via the map $\underline{n} \mapsto W_n$, and that map is first-order definable only through the *recursion theorem*. `Reverse.v` deliberately sidesteps the recursion theorem by iterating on the meta-level `nat`; rendering the same construction first-order means formalizing the recursion theorem's defining formula — a heavy object-level construction, and the genuine first-order content of the reverse direction. Section 10 carries it out.

8 Mechanization III: a proof calculus, soundness, and a cross-theory bridge

So far everything has been semantic (about models). `Calculus.v` builds the *syntactic* layer: an actual proof calculus over `form`, its admissible rules, and a soundness theorem; `Equivalence.v` then draws a first bridge between the two theories at the level of provability.

8.1 A natural-deduction calculus

`Prov G a` means "a is derivable from the context `G` (a list of formulas)." The calculus is classical natural deduction:

```
Inductive Prov : list form -> form -> Prop :=
| P_ass    : forall G a, In a G -> Prov G a
| P_impI   : forall G a b, Prov (a :: G) b -> Prov G (fImp a b)
| P_impE   : forall G a b, Prov G (fImp a b) -> Prov G a -> Prov G b
| P_botE   : forall G a, Prov G fBot -> Prov G a
| P_lem    : forall G a, Prov G (fOr a (fImp a fBot))
| P_andI   : forall G a b, Prov G a -> Prov G b -> Prov G (fAnd a b)
| P_andE1  : forall G a b, Prov G (fAnd a b) -> Prov G a
| P_andE2  : forall G a b, Prov G (fAnd a b) -> Prov G b
| P_orI1   : forall G a b, Prov G a -> Prov G (fOr a b)
| P_orI2   : forall G a b, Prov G b -> Prov G (fOr a b)
| P_orE    : forall G a b c, Prov G (fOr a b) -> Prov (a :: G) c -> Prov (b :: G) c
    -> Prov G c
| P_allI   : forall G a, Prov (map (rename S) G) a -> Prov G (fAll a)
| P_allE   : forall G a k, Prov G (fAll a) -> Prov G (rename (inst k) a)
```

```
| P_exI    : forall G a k, Prov G (rename (inst k) a) -> Prov G (fEx a)
| P_exE    : forall G a c, Prov G (fEx a) -> Prov (a :: map (rename S) G) (rename S
    c) -> Prov G c
| P_eqRefl : forall G k, Prov G (fEq k k)
| P_eqElim : forall G i j a,
    Prov G (fEq i j) -> Prov G (rename (inst i) a) -> Prov G (rename (inst j) a).
```

The De Bruijn discipline shows up in the quantifier rules. In `P_allI` (universal introduction), to prove $\forall a$ we shift the context outward by `rename S` — index 0 becomes the fresh "eigenvariable" bound by the new $\forall$, and the old context variables move up by one. In `P_allE` (universal elimination), instantiating $\forall a$ at variable k is the renaming `rename (inst k) a`, where `inst k` maps the bound index 0 to k and decrements the rest. The existential rules are dual. Crucially, instantiation is *just renaming* — the purely relational signature again paying off — so the same `rename`/`Sat_rename` machinery handles quantifiers with no separate substitution.

8.2 The equality rule: a genuine correctness fix

The two equality rules deserve a word, because getting them right was a real correction during development. The reflexivity rule `P_eqRefl` is obvious. For the elimination rule, a tempting but *too weak* design is a "replace-the-variable-everywhere" congruence: from $i = j$ and φ, infer φ with every occurrence of i rewritten to j. That rule cannot even derive *symmetry* of equality: to get $j = i$ from $i = j$ you would rewrite i to j in the formula $j = i$, but $j = i$ contains no i in the right slot to key on, and the rewrite erases the very occurrence you need.

The fix is the proper **Leibniz** rule `P_eqElim`: a formula `a` is treated as a property with a hole at De Bruijn index 0; from $i = j$ and "`a` with the hole filled by i" (`rename (inst i) a`) we infer "`a` with the hole filled by j" (`rename (inst j) a`). This single rule yields symmetry, transitivity, and full congruence, and — as we verify — it is sound. Without it the completeness theorem of §9 would have been unreachable, since the term model is a quotient by provable equality and needs all the equality reasoning. A small example of the rule earning its keep: it is exactly what makes the term-model congruence lemmas go through.

8.3 Soundness, and the bridge $\mathbf{ZF} \vdash \varphi \Rightarrow \mathbf{T} \models \varphi$

Soundness says the calculus only proves things that are true in every model satisfying the context:

```
Theorem soundness :
  forall G a, Prov G a -> forall e, (forall x, In x G -> Sat e x) -> Sat e a.
```

The proof is one case per rule; the quantifier and equality cases are where `Sat_rename` and `Sat_ext` are spent.

Now encode the ZF axioms as closed formulas (`Ext_form`, `Pair_form`, …, `Reg_form`, with Separation and Replacement as schemas over `form`), collect them into a predicate `ZFax`, and define provability from the ZF axioms:

```
Definition ZFprov (phi : form) : Prop :=
  exists G, (forall x, In x G -> ZFax x) /\ Prov G phi.

Theorem ZF_provable_holds_in_T :
  forall phi, ZFprov phi -> forall e, Sat e phi.
```

Because the ambient section assumes the T-axioms, this theorem says: *in any model of the Closure axiomatization T, every ZF-provable formula holds*, i.e. $\mathsf{ZF} \vdash \varphi \Rightarrow \mathsf{T} \models \varphi$. It is the marriage of syntactic soundness with the forward semantic equivalence: each encoded ZF axiom is checked to hold in the T-model (using the derived `Pairing`, `Union`, `ReplacementFO`, `Infinity` for the four traded ones, and the T-hypotheses for the shared ones), and then `soundness` propagates truth through any ZF derivation.

The symmetric statement $\mathsf{T} \vdash \varphi \Rightarrow \mathsf{ZF} \models \varphi$ needs "every ZF-model satisfies `ClosureFO`" — the deep reverse direction, i.e. the recursion theorem as discussed; it is proved in §10.

9 Mechanization IV: Gödel completeness, compactness, and the syntactic equivalence

`Completeness.v` proves the converse of soundness — **Gödel completeness** — from scratch, then leverages it to obtain compactness for sentence theories; the forward *syntactic* equivalence $\mathsf{ZF} \vdash \varphi \Rightarrow \mathsf{T} \vdash \varphi$ follows in `Equivalence.v`. (`Completeness.v` is fully generic: it applies to any theory over this language, and nothing in it mentions set theory.)

9.1 The goal

```
Theorem completeness :
  forall G phi,
    (forall (Dom : Type) (m : Dom -> Dom -> Prop) (v : nat -> Dom),
       (forall g, In g G -> Sat Dom m v g) -> Sat Dom m v phi) ->
    Prov G phi.

Corollary prov_iff_valid :
  forall G phi,
    Prov G phi <->
    (forall (Dom : Type) (m : Dom -> Dom -> Prop) (v : nat -> Dom),
       (forall g, In g G -> Sat Dom m v g) -> Sat Dom m v phi).
```

In words: *a formula is provable in the calculus if and only if it is valid in every model.* The "only if" is soundness; the "if" is the content of completeness. The standard route is Henkin's: show that a *consistent* set of formulas has a model (then contrapose). We outline the construction as it appears in the file.

9.2 Proof-theory infrastructure

The construction leans on the usual admissible rules, all provided by `Calculus.v`: weakening, proof by contradiction and double-negation elimination, a notion of consistency `Con G := ~ Prov G fBot`, the equality kit (symmetry/transitivity/congruence — available only *because* the equality rule was corrected to the Leibniz `P_eqElim`), the admissibility of renaming `Prov_rename`, and cut `Prov_cut`.

9.3 Model existence: the term model and the truth lemma

The heart is `model_exists`: an abstract *maximal-consistent Henkin* theory `T` (one that decides every formula and provides a witness for every existential) has a model. The model is the *quotient term model*. Since terms are just variables (`nat`), the domain ought to be variables modulo provable

equality `ceq i j`. To get an honest Coq type rather than a setoid, we pick *canonical representatives* with Hilbert's ε:

```
Definition rep (i : nat) : nat := epsilon (inhabits 0) (fun j => ceq i j).
Definition D : Type := { n : nat | rep n = n }.       (* canonical reps *)
Definition memD (x y : D) : Prop := cmem (proj1_sig x) (proj1_sig y).
```

The linchpin is the **truth lemma**: in this model, satisfaction of a formula coincides with its membership in the theory `T`. It is proved by structural induction on the formula, with the substitution `s` generalized: a quantifier case recurses on the *body* of the quantified formula, at a substitution extended by the witness variable (`rename_inst_up` rewrites the instantiated renaming into that form):

```
Lemma truth :
  forall a (s : nat -> nat),
    Sat D memD (fun i => mkD (s i)) a <-> T (rename s a).
```

The atomic cases use the congruence of `cmem`/`ceq` under the representative map; the connective cases use that `T` is maximal-consistent (it respects $\to$, $\wedge$, $\vee$, $\neg$); the quantifier cases use that `T` is Henkin (an existential is in `T` iff some witness instance is, and dually for $\forall$).

9.4 Lindenbaum and Henkin: building the maximal theory

To apply `model_exists` we must *construct* a maximal-consistent Henkin extension of any consistent set. The construction is carried out once, in the generality that the endgame needs: relative to a base theory B of *sentences* — axioms with no free variables, as ZF and T will be — together with a finite working context. Provability and consistency relative to B mean: some finite batch of B-axioms joins the derivation.

```
Definition Sentence (f : form) : Prop := forall n, ~ Free n f.
Definition Sentences (B : form -> Prop) : Prop := forall f, B f -> Sentence f.
Definition BProv (B : form -> Prop) (G : list form) (phi : form) : Prop :=
  exists Gb, (forall x, In x Gb -> B x) /\ Prov (Gb ++ G) phi.
```

The construction needs an enumeration of all formulas; `Fol.v` builds one from Cantor's pairing function and proves it surjective:

```
Definition Enum (n : nat) : form := decode (S n) n.
Lemma Enum_surj : forall f, exists n, Enum n = f.
```

Then a chain `chainB B L0 n` processes formulas one at a time: at each step it decides the next formula (adding it or its negation, by the Lindenbaum step `BCon_cons_or`) and, when it adds an existential, also adds a *fresh-witness* instance (the eigenvariable and Henkin-witness lemmas, from `Prov_rename` plus a freshness analysis of free variables). Freshness is exactly where the sentence restriction earns its keep: the witness variable must be fresh for everything in play, and for an infinite base theory that mentions all variables there would be none — but a witness fresh for the *finite* list of formulas added so far is automatically fresh with respect to the sentence base B, since sentences contribute no free variables to clash with. The limit theory `Tinf` is shown to satisfy all five hypotheses of `model_exists` — consistent, complete, deductively closed, and Henkin for both $\exists$ and $\forall$. Combining:

```
Theorem model_of_BCon :
  forall B L0, Sentences B -> BCon B L0 ->
    exists Dom m v, (forall g, B g -> Sat Dom m v g)
                 /\ (forall g, In g L0 -> Sat Dom m v g).
```

9.5 Completeness, finite and infinite

Finite-context completeness is now the *empty-base* instance $B = \emptyset$: `BProv (fun _ => False) G phi` is plain `Prov G phi` (`BProv_empty`), so `model_of_BCon` specializes to `model_of_con` (every consistent finite context has a model), and contraposing gives `completeness`, hence `prov_iff_valid`. At the other end, taking L_0 to carry a negated goal sentence gives the compactness-style lift to genuinely infinite theories — such as ZF and T, whose Separation/Closure schemas have infinitely many instances:

```
Theorem completeness_inf :
  forall B psi, Sentences B -> Sentence psi ->
    (forall Dom m v, (forall g, B g -> Sat Dom m v g) -> Sat Dom m v psi) ->
    BProv B nil psi.
```

and, as an immediate corollary, the model-theoretic characterization of deductive equivalence:

```
Theorem theory_equiv :
  forall B1 B2, Sentences B1 -> Sentences B2 ->
    (forall Dom m v, (forall g, B1 g -> Sat Dom m v g) <-> (forall g, B2 g -> Sat Dom
    m v g)) ->
    forall psi, Sentence psi -> (BProv B1 nil psi <-> BProv B2 nil psi).
```

`theory_equiv` is the abstract engine behind the whole article: *two sentence theories with the same models prove the same sentences.*

9.6 ZF and T as sentence theories, and ZF $\vdash \varphi \Rightarrow$ T $\vdash \varphi$

To feed ZF and T into this machinery we encode them as sentence theories. Each axiom is universally closed by `seal` (close every free variable with $\forall$), so the parametrized schema instances become genuine sentences; crucially, the Closure schema is encoded as a closed formula `Closure_form` whose `Sat`-meaning is bridged back to the abstract `SetLike`/closure statement (`bridge_Closure` and friends):

```
Definition seal (f : form) : form := closeN (bound f) f.     (* universal closure *)
Definition Tax_s  (f : form) : Prop := (* sealed Ext, Reg, Pow, Sep-schema,
    Closure-schema *)
Definition ZFax_s (f : form) : Prop := (* sealed Ext, Reg, Pow, Pair, Union, Inf,
    Sep-schema, Repl-schema *)
```

The semantic content of the forward equivalence is repackaged as: *every model of T is a model of ZF.*

```
Lemma Tmodel_sat_ZF :
  forall (V : Type) (mem : V -> V -> Prop) v,
    (forall g, Tax_s g -> Sat V mem v g) -> (forall g, ZFax_s g -> Sat V mem v g).
```

The proof extracts the abstract axioms from a T-model through the `bridge_*` lemmas, then reuses the derived satisfaction facts of the forward trade (`sat_Pair`, `sat_Union`, `sat_Inf`, `sat_Repl`) to discharge the four traded ZF axioms. Soundness plus `completeness_inf` then deliver the forward *syntactic* direction:

```
Theorem ZF_implies_T :
  forall phi, Sentence phi -> BProv ZFax_s nil phi -> BProv Tax_s nil phi.
```

Everything ZF proves, T proves. Read against `theory_equiv`, this is one half of the statement that ZF and T are the same theory — the half that goes through cleanly, because `Tmodel_sat_ZF` could reuse the deep forward derivations wholesale. The other half needs the dual model inclusion, and that is where the recursion theorem finally has to be paid for.

10 Mechanization V: the deep reverse — the recursion theorem as a first-order formula

`Zf.v` and `Equivalence.v` close the converse. By `theory_equiv` (or directly by `completeness_inf` plus soundness), the syntactic statement $\mathsf{T} \vdash \varphi \Rightarrow \mathsf{ZF} \vdash \varphi$ follows from the dual of `Tmodel_sat_ZF`: *every first-order ZF-model satisfies every instance of* `Closure_form`. That is the *deep reverse* direction, and it is genuinely hard, for a reason that has been visible since §7.

10.1 The obstacle, and the shape of the solution

`Reverse.v` proves Closure using a *second-order* ingredient: the iteration W_n lives on the metatheory's `nat`, and the family $\{W_n\}$ is collected by an external recursion. A first-order ZF-model is not handed such a meta-recursion — a nonstandard model has stages at nonstandard levels that no meta-level `nat` ever reaches — so it must *build* the family internally, as a set-coded function. Doing that is precisely the content of the **finite recursion theorem** on ω:

- encode functions as sets of Kuratowski pairs;
- formalize the "m-approximation" of the iteration and prove existence and uniqueness of approximations by ω-induction, with every set-existence step passing through the model's *first-order* Separation and Replacement — i.e. through genuine formulas;
- feed the (now provably functional) stage relation to first-order Replacement over ω and take the union.

One design decision deserves emphasis. We do *not* write raw object derivations in the calculus `Prov`; we prove *satisfaction in an arbitrary first-order model*, quantifying over models inside Coq, and let the completeness theorem of §9 convert model-theoretic truth into provability. This is the standard model-theoretic route, and it is what the completeness layer was built for — but nothing conceptual is skipped by it: the recursion theorem's *defining formula* still has to be constructed, as an actual De Bruijn `form`, because the model's Separation and Replacement schemas accept nothing else. The heavy lifting is exactly where the textbook says it is; only the final conversion from truth to proof is outsourced to Gödel.

10.2 An internal ω and a definable-induction schema

Fix a model $(V, \in)$ of the first-order ZF axioms (the file extracts them from `ZFax_s` through `bridge_*` lemmas; notably the list is {Ext, Sep, Pair, Union, Inf, Repl} — *neither Powerset nor Regularity is needed*, so the deep reverse inherits the sharpening of §3.3 at the first-order level). Inside V the file rebuilds the basic algebra — pairs, unions, successors `vsucc`, and Kuratowski pairs `kpair` with machine-checked injectivity `kpair_inj` — and then carves the internal ω out of the Infinity witness *by a formula*: ω is the set of members of the inductive witness that belong to every inductive set, where "inductive" is itself a `form` (`fInd`). The payoff is the **definable-induction schema**, the engine that replaces the meta-level `nat`:

```
Lemma omega_ind :
  forall (phi : form) (e : nat -> V),
    SAT (SC vempty e) phi ->
    (forall n, n  omega -> SAT (SC n e) phi -> SAT (SC (vsucc n) e) phi) ->
    forall n, n  omega -> SAT (SC n e) phi.
```

Induction holds for *any property expressible by a formula with parameters* — and only for those, which is exactly the discipline the first-order setting imposes. The internal arithmetic needed later (transitivity of ω and of each natural, irreflexivity `nat_no_self`, successor injectivity `succ_inj_nat`) is proved through `omega_ind` with small explicit formulas, Foundation-free, mirroring the numeral lemmas of `Reverse.v` one level down.

10.3 The recursion theorem's defining formula

The construction is parameterized by the closure relation: a formula `psiC` with argument slots $0, 1$ and parameters `eC` (the instance of the schema being proved). Two preliminary canonicalizations tame the choice functions that `Reverse.v` used freely:

- the *canonical predecessor set* $\mathrm{predSet}(v) = \{z : z \prec v\}$ (`predSet`) — carved by Separation inside an ε-chosen bound from set-likeness, but unique by Extensionality, so the ε leaks nothing into the object level; its graph "$y = \mathrm{predSet}(x)$" is a formula (`psiPS`), provably functional, so first-order Replacement can push it through a set;
- the canonical one-step operator $\mathrm{gstep}(t) = t \cup \bigcup\{\mathrm{predSet}(v) : v \in t\}$, with "$y = \mathrm{gstep}(t)$" again a formula (`fStepF`).

A library of formula macros with satisfaction specs (`fEmptyF`, `fSingF`, `fUPairF`, `fKPairF`, `fPairMemF`, `fSuccF`, …) then assembles the **approximation predicate**: $\mathrm{Approx}(f, m)$ (`Approx`) says f is (the graph of) a function on $\{0, \ldots, m\}$ recording the stages s, $g(s)$, $g(g(s)), \ldots$ — five clauses: functionality, domain bound, the base pair $\langle 0, s\rangle \in f$, domain coverage, and the recurrence $f(k{+}1) = g(f(k))$. Each clause is both a Coq predicate and a `form` (`fAppC1`–`fAppC5`, packaged as `fApproxF`), with a spec lemma equating the two readings. This `fApproxF` — roughly the formula $\exists f\,[\mathrm{Fun}(f) \wedge \mathrm{dom}\, f = m{+}1 \wedge f(0) = s \wedge \forall k{<}m\ f(k{+}1) = g(f(k))]$ of the textbook proof — *is* the recursion theorem's defining formula, built for real.

Existence and agreement of approximations are proved by `omega_ind` (existence extends an m-approximation by one new Kuratowski pair, where injectivity `kpair_inj` and the arithmetic lemmas guarantee the extension stays functional; agreement runs a second internal induction with f, f' as parameters). Agreement is the crucial one: it makes the **stage relation**

$$\Theta(y, m) \ :\equiv\ m \in \omega \wedge \exists f\,(\mathrm{Approx}(f, m) \wedge \langle m, y\rangle \in f)$$

functional (`theta_functional`) — exactly the premise first-order Replacement demands. Replacement applied to Θ (as the formula `fThetaF`) over ω collects the stages; the union of the image is the closure set:

```
Theorem ClosureFO_of_ZF :  (* inside an arbitrary FO ZF-model *)
  forall psiC eC, SetLike (relOf psiC eC) ->
    forall s, exists w, Sub s w /\
      (forall u v, relOf psiC eC u v -> v  w -> u  w).
```

10.4 The converse, and the equivalence

Bridging back through `Closure_form` gives `ZFmodel_sat_Closure` (every ZF-model satisfies every sealed Closure instance), hence `ZFmodel_sat_T` — every ZF-model is a T-model, the dual of `Tmodel_sat_ZF`. Soundness plus `completeness_inf` then deliver the converse syntactic direction, and with `ZF_implies_T` the headline:

```
Theorem T_implies_ZF :
  forall phi, Sentence phi -> BProv Tax_s nil phi -> BProv ZFax_s nil phi.

Theorem T_iff_ZF :
  forall phi, Sentence phi -> (BProv Tax_s nil phi <-> BProv ZFax_s nil phi).
```

The two theories also provably have *the same models* (`T_ZF_same_models`), so `T_iff_ZF` falls out a second time as an instance of the abstract `theory_equiv` — a pleasant cross-check that the general machinery and the concrete construction agree (`T_iff_ZF_via_theory_equiv`).

11 What is proven

Statement	Status	File
$\mathsf{T} \models$ Pairing, Union, Replacement, Infinity	machine-checked	`Forward.v`
$\mathsf{ZF} \models$ Closure (set-like relations)	machine-checked	`Reverse.v`
forward trade from *first-order* schemas	machine-checked	`Equivalence.v`
sound ND calculus; $\mathsf{ZF} \vdash \varphi \Rightarrow \mathsf{T} \models \varphi$	machine-checked	`Calculus.v`, `Equivalence.v`
truth lemma / model existence	machine-checked	`Completeness.v`
completeness $\mathrm{Prov}\, G\, \varphi \Leftrightarrow G \models \varphi$	machine-checked	`Completeness.v`
compactness ($B \models \varphi \Rightarrow B \vdash \varphi$, sentences)	machine-checked	`Completeness.v`
same-model sentence theories are deductively equal	machine-checked	`Completeness.v`
$\mathsf{ZF} \vdash \varphi \Rightarrow \mathsf{T} \vdash \varphi$	machine-checked	`Equivalence.v`
deep reverse: every FO ZF-model satisfies Closure	machine-checked	`Zf.v`
$\mathsf{T} \vdash \varphi \Rightarrow \mathsf{ZF} \vdash \varphi$	machine-checked	`Equivalence.v`
$\mathsf{T} \vdash \varphi \Leftrightarrow \mathsf{ZF} \vdash \varphi$	machine-checked	`Equivalence.v`

The table is closed everywhere: both semantic directions, both syntactic directions, and the logical infrastructure (soundness, completeness, compactness, deductive equivalence of same-model theories) that connects them.

11.1 Trust: axioms used, no admitted goals

Running `Print Assumptions` on the headline theorems reports only the standard axioms of classical mathematics, and nothing else:

```
classic                                (* excluded middle in Prop *)
constructive_indefinite_description    (* Hilbert epsilon / description *)
functional_extensionality
propositional_extensionality
proof_irrelevance
```

There is no `Admitted` and no custom `Axiom` anywhere in the development. These five are exactly the principles one expects when mechanizing classical set theory with a quotient term model: classical logic, a description operator to turn existence axioms into operations, and the extensionality/irrelevance principles that make the quotient an honest type.

A word on `constructive_indefinite_description` (with `epsilon`), since it is easy to misread. It is *metatheoretic* choice — Hilbert's ε in the ambient logic — and lives at a different level from the set-theoretic Axiom of Choice, which (as §2 explained) appears in neither T nor ZF here, and which these certificates accordingly do not list. Moreover the two are not even the same *kind* of principle. Hilbert's ε is a uniform, definable selector: it returns a canonical witness for every existence statement at once. In the set-theoretic reading that is the *global* form of choice — $\varepsilon x.(x \in a)$ is a global choice function, picking from every nonempty set — rather than the set-by-set Axiom of Choice; the two are inter-definable over a base set theory, and global choice is stronger in character than AC (though conservative over ZFC). So the metatheory is, if anything, helping itself to *more* choice than the object theories ever mention — but only ever in the harmless *descriptive* way: ε is used solely to *name* witnesses already proved to exist (turning the existential axioms into the operators `power`, `sep`, `host`, …) and to pick canonical representatives for the quotient term model. It never proves a choice statement that was not already present, and it adds no object-level set-theoretic strength. Beyond these standard classical axioms the development assumes no set-theoretic universe or large-cardinal strength.

12 Conclusion

Two punchlines, one mathematical and one about formalization.

The mathematics. Four of ZF's axioms — Pairing, Union, Infinity, Replacement — are not four independent ideas but four instances of *one*: collect the predecessors of a seed under a set-like relation. The single Closure schema packages that idea, and over the structural base it is interderivable with all four. The non-obvious lesson is *where the power comes from*: not from Closure alone but from Closure resting on *hosting* — the one-line fact that every set is a member of some set. That fact makes "has a definable predecessor" imply "is set-like" for free, which is what turns a statement about transitive closures into general collection. And the proof assistant pins down exactly how much is doing the work, by reading off which hypotheses each derivation discharged. The verdict is bracing: of the "structural" axioms, *none* is used in full. Regularity and

Choice are passengers, used in neither direction; Powerset is used only through its trivial hosting shadow $\forall a\, \exists y\, (a \in y)$, and only forward (the reverse needs no Powerset at all). The machine-checked $\mathsf{ZF} - \text{Powerset} - \text{Regularity} \vdash \text{Closure}$, and the host-only forward derivations, make this precise: the genuine engine of the whole equivalence is just {Extensionality, Separation, Closure} together with hosting. What looked at first like a Powerset/Regularity mirror was an artifact of a convenient proof.

The formalization. We did not stop at the equivalence. The development ascends from a shallow model argument, to a deep embedding with genuinely first-order schemas, to a sound and *complete* proof calculus built from scratch (truth lemma, Lindenbaum/Henkin, Gödel completeness), to compactness for sentence theories, to the abstract theorem that same-model sentence theories are deductively equivalent — and finally to the deep reverse: the finite recursion theorem built as an actual first-order formula (Kuratowski pairs, an internal ω with a definable-induction schema, the approximation predicate and its functionality) and run inside an arbitrary first-order ZF-model, so that Gödel completeness converts model-theoretic truth into the converse $\mathsf{T} \vdash \varphi \Rightarrow \mathsf{ZF} \vdash \varphi$. The result is the full machine-checked deductive equivalence $\mathsf{T} \vdash \varphi \Leftrightarrow \mathsf{ZF} \vdash \varphi$, with no admitted goals and only the standard classical axioms — a complete and honest account of what it takes to certify that a slogan ("one closure schema") really is ZF in disguise.

A File inventory and build instructions

The development is seven Rocq/Coq files organized by topic across `Logic/FirstOrder/Coq/`, `SetTheory/ZF/Coq/`, and `SetTheory/ClosureAxiomatization/Coq/`: three files of generic first-order logic (usable for any theory over the language of one binary relation and equality), one file of first-order ZF, one file for the axiomatization T and the equivalence, and the two original self-contained shallow files. The import DAG:

$$\texttt{Fol} \to \texttt{Calculus} \to \texttt{Completeness}, \qquad \{\texttt{Fol}, \texttt{Calculus}\} \to \texttt{Zf}, \qquad \text{all} \to \texttt{Equivalence}.$$

File	Contents
`Forward.v`	shallow embedding; {Ext, Sep, Closure} + hosting derive Pairing/Union/Replacement/Infinity; linchpin `host`/`host_spec` (Powerset enters only via `powerset_gives_hosting`); dependency-audit `Check`s show the four free of Powerset.
`Reverse.v`	shallow embedding; ZF derives Closure via the transitive-closure recursion; numerals distinct Foundation-free (`onat_trans`, `onat_no_self`, `onat_inj`); audit shows *both* Powerset and Regularity unused.
`Fol.v`	*generic*: deep embedding of first-order logic over $\{\in, =\}$ (`form`, `rename`, `Free`, `seal`, the formula enumeration `Enum`, Tarski `Sat`, `Sat_ext`/`Sat_rename`, `relOf`).
`Calculus.v`	*generic*: classical ND calculus `Prov`, admissible rules (weakening, cut, deduction, `Prov_rename`, the equality kit, Henkin cores), `soundness`.
`Completeness.v`	*generic*: from-scratch Gödel `completeness`/`prov_iff_valid`; `completeness_inf` (compactness for sentence theories); `theory_equiv`.
`Zf.v`	first-order ZF: the axioms as formulas, `ZFax_s`, extraction bridges; internal model mathematics — set algebra with `kpair_inj`, internal ω and the definable-induction schema `omega_ind`, the formula-macro library, `Approx`/`fApproxF` with existence and agreement, `theta_functional`, `ClosureFO_of_ZF`.
`Equivalence.v`	the axiomatization T: deep forward trade; `ZF_provable_holds_in_T`; `Closure_form`; `Tax_s`; `Tmodel_sat_ZF`/`ZFmodel_sat_T`; `ZF_implies_T`; `T_implies_ZF`; `T_iff_ZF`.

`Forward.v` and `Reverse.v` are independent (no inter-file `Require`) and need only the standard library; the library files import along the DAG above. `Completeness.v` additionally uses the standard classical/extensionality modules for the quotient term model.

```
# developed against Rocq 9.0.1
coqc SetTheory/ClosureAxiomatization/Coq/Forward.v
coqc SetTheory/ClosureAxiomatization/Coq/Reverse.v
# From the repository root, build the library in dependency order:
coqc -Q Logic/FirstOrder/Coq FirstOrder Logic/FirstOrder/Coq/Fol.v
coqc -Q Logic/FirstOrder/Coq FirstOrder Logic/FirstOrder/Coq/Calculus.v
coqc -Q Logic/FirstOrder/Coq FirstOrder Logic/FirstOrder/Coq/Completeness.v
coqc -Q Logic/FirstOrder/Coq FirstOrder -Q SetTheory/ZF/Coq ZF \
  SetTheory/ZF/Coq/Zf.v
coqc -Q Logic/FirstOrder/Coq FirstOrder -Q SetTheory/ZF/Coq ZF \
  -Q SetTheory/ClosureAxiomatization/Coq ClosureAxiomatization \
  SetTheory/ClosureAxiomatization/Coq/Equivalence.v
```